# What is scrumpy?

it was authentic scrumpy, rather sharp and very strong

we could taste a famous local product — scrumpy

spending hours in the pub drinking scrumpy

Cornish Scrumpy Medium Dry. £19.28 - Case

# What is scrumpy?

it was authentic scrumpy, rather sharp and very strong

we could taste a famous local product — scrumpy

spending hours in the pub drinking scrumpy

Cornish Scrumpy Medium Dry. £19.28 - Case

# What is scrumpy?

it was authentic scrumpy, rather sharp and very strong

we could taste a famous local product — scrumpy

spending hours in the pub drinking scrumpy

Cornish Scrumpy Medium Dry. £19.28 - Case

# What is scrumpy?

it was authentic scrumpy, rather sharp and very strong

we could taste a famous local product — scrumpy

spending hours in the pub drinking scrumpy

Cornish Scrumpy Medium Dry. £19.28 - Case

# Scrumpy


WESTONS
CIDERS INSPIRED BY TRADITION
SCRUMPY
CLOUDY CIDER
Alc 7.5% Vol

# Natural Language Processing 1

## Lecture 5: Lexical semantics and word embeddings

Martha Lewis
based on slides from Katia Shutova, Dan Jurafsky, James Martin

ILLC
University of Amsterdam

# Outline.

1 Introduction

2 Distributional semantics

3 Semantics with dense vectors

4 Properties of embeddings

# Semantics

Compositional semantics:

- studies how meanings of phrases are constructed out of the meaning of individual words
- principle of compositionality: meaning of each whole phrase derivable from meaning of its parts
- sentence structure conveys some meaning: obtained by syntactic representation

Lexical semantics:

- studies how the meanings of individual words can be represented and induced

# What is lexical meaning?

- recent results in psychology and cognitive neuroscience give us some clues, but we don't have the whole picture yet
- different representations proposed, e.g.
  - formal semantic representations based on logic [4],
  - regions of a conceptual space [2],
  - networks (or frames) relating words to each other [1],
  - distributional representations in statistical NLP (this lecture!)
- but none of the representations gives us a complete account of lexical meaning

# How to approach lexical meaning?

- Formal semantics: set-theoretic approach
  e.g., cat′: the set of all cats; bird′: the set of all birds.
- meaning postulates, e.g.

$$\forall x[\text{bachelor}'(x) \rightarrow \text{man}'(x) \wedge \text{unmarried}'(x)]$$

- Limitations, e.g. *is the Pope a bachelor?*
- Defining concepts through enumeration of all of their features is impossible in practice!
- How would you define e.g. *chair, tomato, thought, democracy*? – impossible for most concepts
- Prototype theory offers an alternative to set-theoretic approaches

# How to approach lexical meaning?

- Formal semantics: set-theoretic approach
  e.g., cat′: the set of all cats; bird′: the set of all birds.
- meaning postulates, e.g.

$$\forall x[\text{bachelor}'(x) \rightarrow \text{man}'(x) \wedge \text{unmarried}'(x)]$$

- Limitations, e.g. *is the Pope a bachelor?*
- Defining concepts through enumeration of all of their features is impossible in practice!
- How would you define e.g. *chair, tomato, thought, democracy*? – impossible for most concepts
- Prototype theory offers an alternative to set-theoretic approaches

# What do we want from a theory of meaning?

**Prototype theory** [6]

- introduced the notion of graded semantic categories
- no clear boundaries; no requirement that a property be shared by all members
- certain members of a category are more central or prototypical (i.e. instantiate the prototype)
  *furniture: chair is more prototypical than stool*
- Categories form around prototypes; new members added on basis of resemblance to prototype

# What do we want from a theory of meaning?

**Semantic relations**

Hyponymy: IS-A

*dog* is a hyponym of *animal*
*animal* is a hypernym of *dog*

- hyponymy relationships form a taxonomy
- works best for concrete nouns

# What do we want from a theory of meaning?

**Semantic relations**

Synonymy e.g., *aubergine*/*eggplant*.

Antonymy e.g., *big*/*little*

Meronomy: PART-OF e.g., *arm* is a meronym of *body*, *steering wheel* is a meronym of *car*

Also:

Near-synonymy/similarity e.g., *exciting*/*thrilling*
e.g., *slim*/*slender*/*thin*/*skinny*

**WordNet**[1]: a large-scale lexical resource linking words by their semantic relations.

[1] https://wordnet.princeton.edu/, https://en-word.net/

# Psychological accounts of similarity

- We already saw protoype theory
- ...but this gives no indication of how to actually *represent* concepts.
- Other theories propose representing items as points in a feature space [7], with similarity between items inversely related to distance between items
- ...or as collections of features, where similarity depends on which features are shared [8]

# Polysemy and word senses

The children **ran** to the store
If you see this man, **run**!
Service **runs** all the way to Cranbury
She is **running** a relief operation in Sudan
the story or argument **runs** as follows
Does this old car still **run** well?
Interest rates **run** from 5 to 10 percent
Who's **running** for treasurer this year?
They **ran** the tapes over and over again
These dresses **run** small

# Summary

- Lexical semantics studies how the meanings of individual words can be represented and induced.
- There are a number of different theories for how best to represent words: formal semantics, conceptual spaces, frames, distributions among others.
- Word meanings have properties such as propotypicality
- ...and relations such as hyponymy, synonymy/antonymy, meronymy, and similarity
- In psychology, similarity is often related to distance between points in feature space
- Finally, words and concepts are not one-to-one: there are many ways to describe a given concept with a word, and one word can describe many (often related) concepts

# Outline.

1 Introduction

2 Distributional semantics

3 Semantics with dense vectors

4 Properties of embeddings

# Distributional semantics

Can we build a theory of how to represent word meaning, that accounts for at least some of these aspects of language?
We'll introduce distributional semantics, where words are represented by distributional vector embeddings.

- The standard model in language processing!
- Handles many of our goals!

# Distributional hypothesis

*You shall know a word by the company it keeps* (Firth)

*The meaning of a word is defined by the way it is used* (Wittgenstein).

it was authentic scrumpy, rather sharp and very strong

we could taste a famous local product — scrumpy

spending hours in the pub drinking scrumpy

Cornish Scrumpy Medium Dry. £19.28 - Case

# Scrumpy


WESTONS
CIDERS INSPIRED BY TRADITION
SCRUMPY
CLOUDY CIDER
Alc 7.5% Vol

# Idea 1: Defining meaning by linguistic distribution

This leads to the distributional hypothesis about word meaning:

- the context surrounding a given word provides information about its meaning;
- words are similar if they share similar linguistic contexts;

# Idea 2: Meaning as a point in a multidimensional space

Borrowing ideas from psychology [7, 8], we can represent a meaning as a point in a feature space.

- Values on feature dimensions contribute to the meaning of the word
- Similarity is calculated as a function of distance in the feature space

## Two ideas

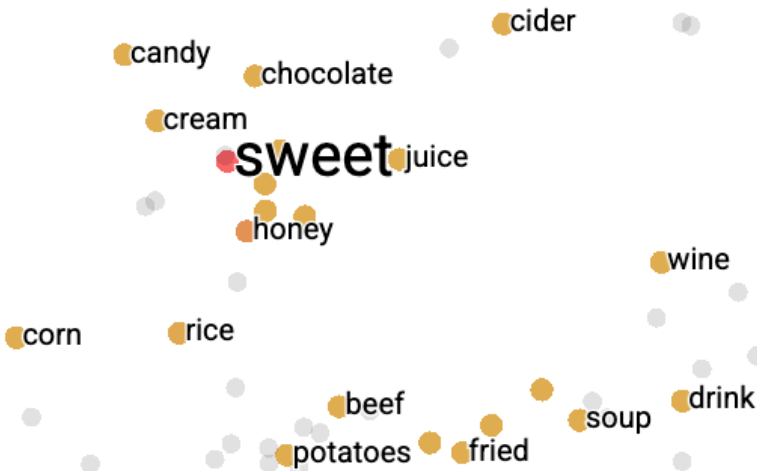
Idea 1: Defining meaning by linguistic distribution

Idea 2: Meaning as a point in multidimensional space

Vector semantics brings these two ideas together.

# The general intuition

- **Words** are vectors in a multidimensional semantic space.
- The **semantic space** has dimensions which correspond to possible contexts – features.
- Entries of the vectors are defined by the distribution of words they occur close to.
- *scrumpy* [...pub 0.8, drink 0.7, strong 0.4, joke 0.2, mansion 0.02, zebra 0.1...]



# We define the meaning of a word as a vector

- Called an “embedding” because it’s embedded into a space
- This is now the standard way to represent meaning in NLP

Every modern NLP algorithm uses embeddings as the representation of word meaning

# The notion of context

How do we define the entries of our vectors? We will use the notion of context.

- We build up a word-context matrix.
- Each entry of the matrix counts how often each word occurs with its context.

| | aardvark | abacus | adept | agate | ... | zydeco |
|---|---|---|---|---|---|---|
| aardvark | | | | | | |
| abacus | | | | | | |
| adept | | | | | | |
| agate | | | | | | |
| ... | | | | | | |
| zydeco | | | | | | |

How often does agate occur near abacus?

# Example

*A cat chased a mouse. A dog chased the cat. The cat bit the mouse. The dog bit the cat. The dog chased the mouse and the car. The car needs fuel.*

Words: cat chased mouse dog bit car needs fuel

# Example

Removing stopwords (in this case the, a, and):

> *cat chased mouse. dog chased cat. cat bit mouse. dog bit cat. dog chased mouse and car. car needs fuel.*

Words: cat chased mouse dog bit car needs fuel

| | cat | chased | mouse | dog | bit | car | needs | fuel |
|---|---|---|---|---|---|---|---|---|
| cat | | | | | | | | |
| chased | | | | | | | | |
| mouse | | | | | | | | |
| dog | | | | | | | | |
| bit | | | | | | | | |
| car | | | | | | | | |
| need | | | | | | | | |
| fuel | | | | | | | | |

# Example

Removing stopwords (in this case the, a, and):

> *cat chased mouse. dog chased cat. cat bit mouse. dog bit cat. dog chased mouse and car. car needs fuel.*

Words: cat chased mouse dog bit car needs fuel

| | cat | chased | mouse | dog | bit | car | needs | fuel |
|---|---|---|---|---|---|---|---|---|
| cat | 0 | 2 | 2 | 2 | 2 | 0 | 0 | 0 |
| chased | 2 | 0 | 2 | 3 | 0 | 1 | 0 | 0 |
| mouse | 2 | 2 | 0 | 1 | 1 | 1 | 0 | 0 |
| dog | 2 | 3 | 1 | 0 | 1 | 1 | 0 | 0 |
| bit | 2 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| car | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| needs | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| fuel | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |

# Example

Take the rows as vectors representing the word:

$$cat = [0, 2, 2, 2, 2, 0, 0, 0]$$
$$dog = [2, 3, 1, 0, 1, 1, 0, 0]$$
$$fuel = [0, 0, 0, 0, 0, 1, 1, 0]$$

We see that cat and dog overlap in more contexts than cat and fuel: they are more similar. We can measure similarity using cosine similarity or inner product.

# The notion of context

1. **Word windows**: *n* words on either side of the lexical item.
   Example: n=2 (5 words window):
   *| The prime **minister** acknowledged the | question.*

   *minister* [ the 2, prime 1, acknowledged 1, question 0 ]

2. **Lexeme window**: as above but using stems.
   Example: n=2 (5 words window):
   *| The prime **minister** acknowledged the | question.*

   *minister* [ the 2, prime 1, acknowledge 1, question 0 ]

# Context

3 **Syntactic relations** (dependencies). Context for a lexical item is the syntactic dependency structure it belongs to. Example:
*The prime **minister** acknowledged the question.*

*minister* [ prime 1, acknowledge 1]

*minister* [ prime_mod 1, acknowledge_subj 1]

# Context weighting

1. **Binary model**: if context $c$ co-occurs with word $w$, value of vector $\vec{w}$ for dimension $c$ is 1, 0 otherwise.
2. **Basic frequency model:** the value of vector $\vec{w}$ for dimension $c$ is the number of times that $c$ co-occurs with $w$.
3. **Characteristic model**: Weights given to the vector components express how *characteristic* a given context is for word $w$.
   Examples: positive pointwise mutual information (PPMI), logDice.

# What semantic space?

- Entire vocabulary.
  - + All information included – even rare contexts
  - - Inefficient (100,000s dimensions). Noisy (e.g. *002.png|thumb|right|200px|graph*). Sparse
- Top *n* words with highest frequencies.
  - + More efficient (2000-10000 dimensions). Only 'real' words included.
  - - May miss out on infrequent but relevant contexts.
- Dimensionality reduction using matrix factorization
  - + Very efficient (200-500 dimensions). Captures generalisations in the data.
  - - The resulting matrices are not interpretable.

# An example noun

- *language*:

WORD SKETCH English Wikipedia

Get more space

language as noun 395,083× Combined relations ×

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **speak** verb | verbs with "language" as object | 16975 | 9.9 |
| **programming** noun | modifiers of "language" | 9128 | 9.3 |
| **English** adjective | modifiers of "language" | 7157 | 8.5 |
| **official** adjective | modifiers of "language" | 5667 | 8.4 |
| **foreign** | modifiers of "language" | 4831 | 8.2 |
| **translate** verb | ... into "language" | 4363 | 8.2 |
| **native** adjective | modifiers of "language" | 4143 | 8.0 |
| **culture** noun | "language" and/or ... | 4681 | 8.0 |
| **literature** noun | "language" and/or ... | 2751 | 7.5 |
| **language** noun | "language" and/or ... | 4114 | 7.4 |

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **learn** verb | verbs with "language" as object | 3030 | 7.4 |
| **other** adjective | modifiers of "language" | 12752 | 7.3 |
| **sign** | modifiers of "language" | 2079 | 7.1 |
| **different** adjective | modifiers of "language" | 3698 | 7.1 |
| **Romance** | modifiers of "language" | 1668 | 7.1 |
| **indigenous** adjective | modifiers of "language" | 1806 | 7.1 |
| **Slavic** | modifiers of "language" | 1661 | 7.1 |
| **modern** | modifiers of "language" | 2626 | 7.0 |
| **Germanic** | modifiers of "language" | 1463 | 6.9 |
| **speaker** noun | ... of "language" | 1609 | 6.8 |

Rows per page: 20 1–20 of 1,520 1 / 76

https://www.sketchengine.eu/

# An example adjective

- *academic*:

WORD SKETCH English Wikipedia

Get more space

academic as adjective 108,548× Combined relations

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **journal** | nouns modified by "academic" | 3610 | 9.3 |
| **excellence** | nouns modified by "academic" | 1340 | 8.5 |
| **peer-reviewed** | "academic" and/or ... | 1213 | 8.4 |
| **institution** | nouns modified by "academic" | 2552 | 8.4 |
| **achievement** | nouns modified by "academic" | 1700 | 8.4 |
| **discipline** | nouns modified by "academic" | 1225 | 8.1 |
| **freedom** | nouns modified by "academic" | 894 | 7.4 |
| **department** | nouns modified by "academic" | 1191 | 7.3 |
| **program** | nouns modified by "academic" | 3033 | 7.3 |
| **study** | nouns modified by "academic" | 1853 | 7.0 |

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **standard** noun | nouns modified by "academic" | 923 | 6.9 |
| **athletic** | "academic" and/or ... | 471 | 6.8 |
| **research** | nouns modified by "academic" | 1407 | 6.8 |
| **curriculum** | nouns modified by "academic" | 470 | 6.8 |
| **staff** | nouns modified by "academic" | 878 | 6.7 |
| **degree** | nouns modified by "academic" | 1145 | 6.7 |
| **year** | nouns modified by "academic" | 8322 | 6.7 |
| **career** | nouns modified by "academic" | 2509 | 6.7 |
| **scholarship** | nouns modified by "academic" | 438 | 6.5 |
| **circle** | nouns modified by "academic" | 461 | 6.5 |

Rows per page: 20 1–20 of 224 1 / 12

# Polysemy

- Distribution for *table*, as obtained from Wikipedia.

WORD SKETCH English Wikipedia

Get more space

table as noun 102,249× Combined relations

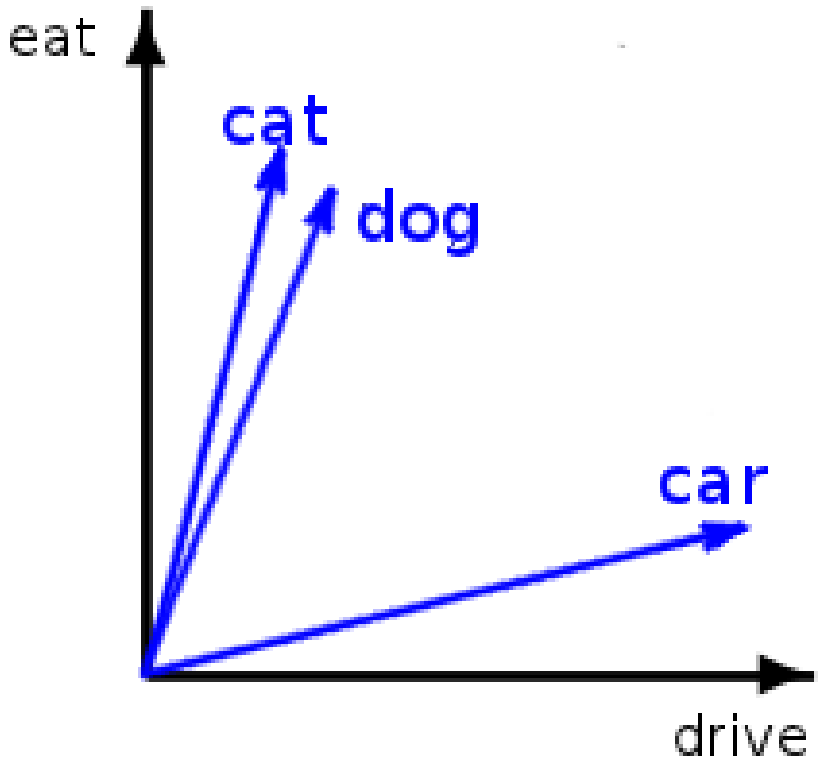
| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **tennis** noun | nouns modified by "table" | 1965 | 8.8 |
| **picnic** | modifiers of "table" | 1272 | 8.5 |
| **periodic** | modifiers of "table" | 950 | 8.1 |
| **bottom** noun | ... of "table" | 838 | 7.6 |
| **chair** noun | "table" and/or ... | 853 | 7.4 |
| **top** noun | ... of "table" | 1040 | 7.3 |
| **hash** | modifiers of "table" | 509 | 7.3 |
| **list** verb | verbs with "table" as subject | 1276 | 7.3 |
| **league** | modifiers of "table" | 1486 | 7.1 |
| **show** verb | verbs with "table" as subject | 2440 | 7.0 |

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **dinner** noun | modifiers of "table" | 487 | 7.0 |
| **dining** | modifiers of "table" | 421 | 6.9 |
| **follow** verb | verbs with "table" as object | 4591 | 6.9 |
| **medal** noun | modifiers of "table" | 770 | 6.9 |
| **top** verb | verbs with "table" as object | 443 | 6.8 |
| **kitchen** noun | modifiers of "table" | 381 | 6.6 |
| **sit** verb | ... at "table" | 603 | 6.6 |
| **water** | modifiers of "table" | 1622 | 6.6 |
| **pool** noun | modifiers of "table" | 457 | 6.6 |
| **content** noun | "table" of ... | 565 | 6.5 |

Rows per page: 20 1–20 of 1,118 1 / 56

# Calculating similarity in a distributional space

- Distributions are vectors, so distance can be calculated.



# Measuring similarity

- Cosine:

$$cos(\theta) = \frac{v}{||v||} \cdot \frac{w}{||w||} = \frac{\sum v_k w_k}{\sqrt{\sum v_k^2} \times \sqrt{\sum w_k^2}} \quad (1)$$

- The cosine measure calculates the angle between two vectors **v** and **w** and is therefore length-independent.
- It is the inner product $\cdot$ between two normalised vectors.
- Other measures include Euclidean distance, city-block distance.

# Example

Recall our word vectors:

$$cat = [0, 2, 2, 2, 2, 0, 0, 0]$$
$$dog = [2, 3, 1, 0, 1, 1, 0, 0]$$
$$fuel = [0, 0, 0, 0, 0, 1, 1, 0]$$

We have: $||cat|| = \sqrt{2^2 + 2^2 + 2^2 + 2^2} = 4$, $||dog|| = 4$, $||fuel|| = \sqrt{2}$.
Cosine similarity between cat and dog is:

$$cos(cat, dog) = \frac{0 \times 2 + 2 \times 3 + 2 \times 1 + 2 \times 0 + 2 \times 1 + 0 \times 1 + 0 \times 0 + 0 \times 0}{4 \times 4}$$
$$= 10/16 = 5/8$$

In contrast, $cos(cat, fuel) = 0$, $cos(dog, fuel) = 1/(4\sqrt{2})$

# The scale of similarity: some examples

house – building 0.43
gem – jewel 0.31
capitalism – communism 0.29
motorcycle – bike 0.29
test – exam 0.27
school – student 0.25
singer – academic 0.17
horse – farm 0.13
man –accident 0.09
tree – auction 0.02
cat –county 0.007

# Words most similar to *cat*

as chosen from the 5000 most frequent nouns in Wikipedia.

| | | | |
|---|---|---|---|
| 1 cat | 0.29 human | 0.25 woman | 0.22 monster |
| 0.45 dog | 0.29 goat | 0.25 fish | 0.22 people |
| 0.36 animal | 0.28 snake | 0.24 squirrel | 0.22 tiger |
| 0.34 rat | 0.28 bear | 0.24 dragon | 0.22 mammal |
| 0.33 rabbit | 0.28 man | 0.24 frog | 0.21 bat |
| 0.33 pig | 0.28 cow | 0.23 baby | 0.21 duck |
| 0.31 monkey | 0.26 fox | 0.23 child | 0.21 cattle |
| 0.31 bird | 0.26 girl | 0.23 lion | 0.21 dinosaur |
| 0.30 horse | 0.26 sheep | 0.23 person | 0.21 character |
| 0.29 mouse | 0.26 boy | 0.23 pet | 0.21 kid |
| 0.29 wolf | 0.26 elephant | 0.23 lizard | 0.21 turtle |
| 0.29 creature | 0.25 deer | 0.23 chicken | 0.20 robot |

# But what is similarity?

- In distributional semantics, very broad notion: synonyms, near-synonyms, hypernyms, taxonomical siblings, antonyms, etc.

| word1 | word2 | similarity |
|---|---|---|
| vanish | disappear | 9.8 |
| behave | obey | 7.3 |
| belief | impression | 5.95 |
| muscle | bone | 3.65 |
| modest | flexible | 0.98 |
| hole | agreement | 0.3 |

Human similarity judgements from SimLex-999 [3]

# Distributional methods are a usage representation

- Distributions are a good conceptual representation if you believe that 'the meaning of a word is given by its usage'.
- Corpus-dependent, culture-dependent, register-dependent.
  Example: similarity between *police* and *cop*: 0.31

# Distribution for *police*

WORD SKETCH English Wikipedia

Get more space

police as noun 206,277× Combined relations ×

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **officer** noun | nouns modified by "police" | 22329 | 10.7 |
| **force** noun | nouns modified by "police" | 10426 | 9.1 |
| **department** noun | nouns modified by "police" | 4020 | 8.6 |
| **station** noun | nouns modified by "police" | 8902 | 8.4 |
| **arrest** verb | verbs with "police" as subject | 2473 | 8.1 |
| **secret** adjective | modifiers of "police" | 2120 | 8.0 |
| **chief** noun | nouns modified by "police" | 1609 | 7.7 |
| **arrest** verb | ... by "police" | 1644 | 7.5 |
| **investigation** noun | nouns modified by "police" | 1499 | 7.4 |
| **detective** noun | nouns modified by "police" | 1040 | 7.2 |

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **brutality** noun | nouns modified by "police" | 893 | 7.1 |
| **riot** noun | modifiers of "police" | 922 | 7.0 |
| **custody** noun | nouns modified by "police" | 900 | 7.0 |
| **arrive** verb | verbs with "police" as subject | 1680 | 7.0 |
| **chief** noun | ... of "police" | 882 | 6.9 |
| **raid** verb | verbs with "police" as subject | 763 | 6.8 |
| **car** | nouns modified by "police" | 1798 | 6.8 |
| **commissioner** noun | nouns modified by "police" | 721 | 6.7 |
| **inspector** noun | nouns modified by "police" | 638 | 6.6 |
| **raid** noun | nouns modified by "police" | 722 | 6.6 |

Rows per page: 20 1–20 of 1,006 1 / 51

# Distribution for *cop*

WORD SKETCH English Wikipedia

Get more space

cop as noun 4,744× Combined relations

| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **undercover** | modifiers of "cop" | 198 | 9.1 |
| **crooked** | modifiers of "cop" | 45 | 7.8 |
| **corrupt** | modifiers of "cop" | 103 | 7.7 |
| **dirty** | modifiers of "cop" | 47 | 6.9 |
| **cop** noun | "cop" and/or ... | 30 | 6.7 |
| **honest** | modifiers of "cop" | 40 | 6.5 |
| **tough** | modifiers of "cop" | 69 | 6.3 |
| **hard-nosed** | modifiers of "cop" | 11 | 6.2 |
| **LAPD** adjective | modifiers of "cop" | 11 | 6.1 |
| **NYPD** | modifiers of "cop" | 11 | 6.1 |

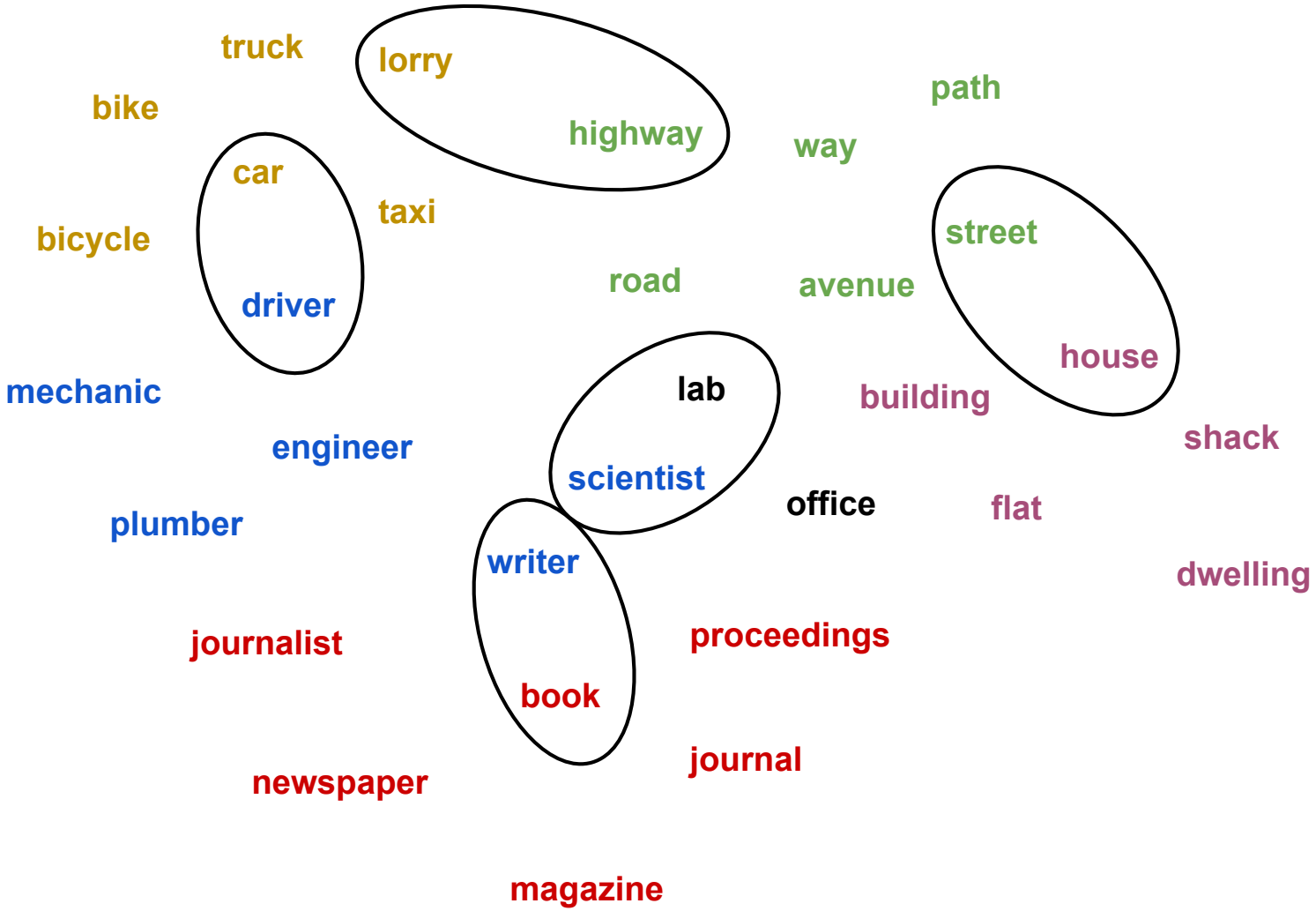
| Word | Grammatical relation | Count | Score ↓ |
|---|---|---|---|
| **disgrace** | verbs with "cop" as object | 13 | 6.0 |
| **rookie** | modifiers of "cop" | 42 | 5.9 |
| **killer** | nouns modified by "cop" | 47 | 5.8 |
| **no-nonsense** | modifiers of "cop" | 9 | 5.7 |
| **beat** noun | modifiers of "cop" | 35 | 5.5 |
| **rogue** | modifiers of "cop" | 13 | 5.5 |
| **narcotic** | modifiers of "cop" | 11 | 5.5 |
| **motorcycle** | modifiers of "cop" | 39 | 5.3 |
| **buddy** | modifiers of "cop" | 8 | 5.1 |
| **wisecrack** | verbs with "cop" as object | 5 | 5.0 |

Rows per page: 20 1–20 of 110 1 / 6

# Clustering nouns



# Clustering nouns



# Summary and key takeaways

## Key takeaways

Word meanings can be represented as vector embeddings that capture their distribution in a corpus of text.

Every modern NLP algorithm uses embeddings as the representation of word meaning.

# Summary and key takeaways

- Vector semantics brings together two ideas: 1) that meaning can be defined via distribution in a corpus and 2) that meanings can be defined by points in a feature space.
- Entries of the vectors are defined by the distribution of words they are close to, and we can choose how to measure this distribution.
- We can also make various choices for how to define the dimensions of the feature space: full vocab, a subset, or reduced dimensionality
- Similarity between words is measured using the cosine of the angle between the word vectors.
- This captures a broad notion of similarity.

# Outline.

1 Introduction

2 Distributional semantics

3 Semantics with dense vectors

4 Properties of embeddings

# Distributional semantic models

1. Count-based models:
   - Explicit vectors: dimensions are elements in the context
   - **long sparse** vectors with **interpretable** dimensions
2. Prediction-based models:
   - Train a model to predict plausible contexts for a word
   - learn word representations in the process
   - **short dense** vectors with **latent** dimensions

# Sparse vs. dense vectors

Why dense vectors?

- easier to use as features in machine learning (less weights to tune)
- may generalize better than storing explicit counts
- may do better at capturing synonymy:
  - e.g. *car* and *automobile* are distinct dimensions in count-based models
  - will not capture similarity between a word with *car* as a neighbour and a word with *automobile* as a neighbour

# Prediction-based distributional models

word2vec: **Skip-gram** model [5]

- inspired by work on neural language models
- train a classifier to predict neighboring words
- learn dense embeddings for the words in the training corpus in the process

# Skip-gram

Instead of counting how often each word w occurs near "apricot"

- Train a classifier on a binary prediction task:
- Is w likely to show up near "apricot"?
- We don't actually care about this task
- But we'll take the learned classifier weights as the word embeddings
- Big idea: self-supervision:
- A word c that occurs near apricot in the corpus counts as the gold "correct answer" for supervised learning
- No need for human labels

# Approach: predict if candidate word c is a “neighbour”

1. Treat the target word t and a neighboring context word c as positive examples.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

# Skip-gram training data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

Goal: Train a classifier that is given a candidate (**w**ord, **c**ontext) pair:

- (apricot, jam)
- (apricot, aardvark)
- ...

And assigns each pair a probability:

$$P(+|w,c)$$
$$P(-|w,c) = 1 - P(+|w,c)$$

# Skip-gram training data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

Goal: Train a classifier that is given a candidate (**w**ord, **c**ontext) pair:

- (apricot, jam)
- (apricot, aardvark)
- ...

And assigns each pair a probability:

$$P(+|w,c)$$
$$P(-|w,c) = 1 - P(+|w,c)$$

# Skip-gram training data

Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

Goal: Train a classifier that is given a candidate (**w**ord, **c**ontext) pair:

- (apricot, jam)
- (apricot, aardvark)
- ...

And assigns each pair a probability:

$$P(+|w, c)$$
$$P(-|w, c) = 1 - P(+|w, c)$$

# Similarity is computed from dot product

- Remember: two vectors are similar if they have a high cosine
- Cosine is just a normalized dot product
- So: $Sim(w, c) \propto w \cdot c$
- We'll need to normalize to get a probability (cosine isn't a probability either)

# Turning dot products into probabilities

$Sim(w, c) \propto w \cdot c$. To turn this into a probability we use the sigmoid from logistic regression.

$$P(+|w, c) = \sigma(w \cdot c) = \frac{1}{1 + \exp(-w \cdot c)}$$

$$P(-|w, c) = 1 - P(+|w, c)$$

$$= \sigma(-w \cdot c) = \frac{1}{1 + \exp(w \cdot c)}$$

# How the skip-gram classifier computes $P(+|w \cdot c)$

$$P(+|w,c) = \sigma(w \cdot c) = \frac{1}{1+\exp(-w \cdot c)}$$

This is for one context word, but we have multiple context words. We assume that they are independent, and multiply together their probabilities:

$$P(+|w,c_{1:L}) = \prod_{i=1}^{L} P(+|w,c_i) = \prod_{i=1}^{L} \sigma(w \cdot c_i) \tag{2}$$

$$\log P(+|w,c_{1:L}) = \sum_{i=1}^{L} \log \sigma(w \cdot c_i) \tag{3}$$

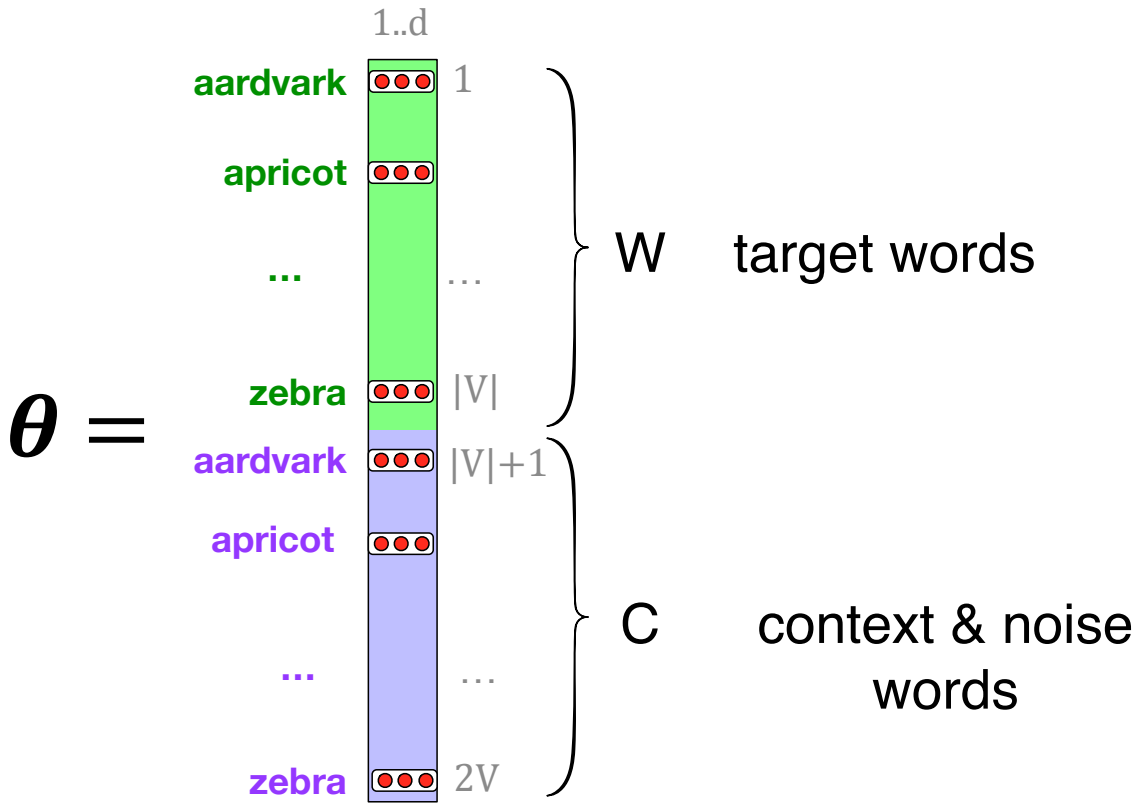
# Skip-gram classifier: summary

A probabilistic classifier, given

- a test target word $w$
- its context window of $L$ words $c_{1:L}$

Estimates probability that w occurs in this window based on similarity of $w$ (embeddings) to $c_{1:L}$ (embeddings).
To compute this, we just need embeddings for all the words.

# Embeddings needed: a set for *w* and a set for *c*




Note that we can stack the *w* embeddings into a marix *W* and the *c* embeddings into a matrix *C*

# Skip-gram with negative sampling

- Objective in training:
  - Make the word like the context words
    lemon, a [tablespoon of apricot preserves or] jam.

    $c_1$ $c_2$ $w$ $c_3$ $c_4$
  - And not like the $k$ negative examples

[cement idle dear coaxial apricot attendant whence forever puddle]

$n_1$ $n_2$ $n_3$ $n_4$ $w$ $n_5$ $n_6$ $n_7$ $n_8$

# Skip-gram with negative sampling: Training examples

Convert the dataset into word pairs:

- **Positive (+)**

  (apricot, tablespoon)
  (apricot, of)
  (apricot, jam)
  (apricot, or)

- **Negative (-)**

  (apricot, cement)
  (apricot, idle)
  (apricot, attendant)
  (apricot, dear)
  ...

# Training

Given the set of positive and negative training instances, and an initial set of embedding vectors The goal of learning is to adjust those word vectors such that we:

- Maximize the similarity of the target word, context word pairs $(w, c_{pos})$ drawn from the positive data
- Minimize the similarity of the $(w, c_{neg})$ pairs drawn from the negative data.

# Loss function for one $w$ with $c_{pos}$, $c_{neg_1}$, ...$c_{neg_k}$

Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the k negative sampled non-neighbour words.

$$
\begin{aligned}
L &= -\log\left(P(+|w, c_{pos}) \prod_{i=1}^{k} P(-|w, c_{neg_i})\right) \\
&= -\log P(+|w, c_{pos}) - \sum_{i=1}^{k} \log P(-|w, c_{neg_i}) \\
&= -\log P(+|w, c_{pos}) - \sum_{i=1}^{k} \log 1 - P(+|w, c_{neg_i}) \\
&= -\log \sigma(w \cdot c_{pos}) - \sum_{i=1}^{k} \log \sigma(w \cdot c_{neg_i})
\end{aligned}
$$

# Learning the classifier

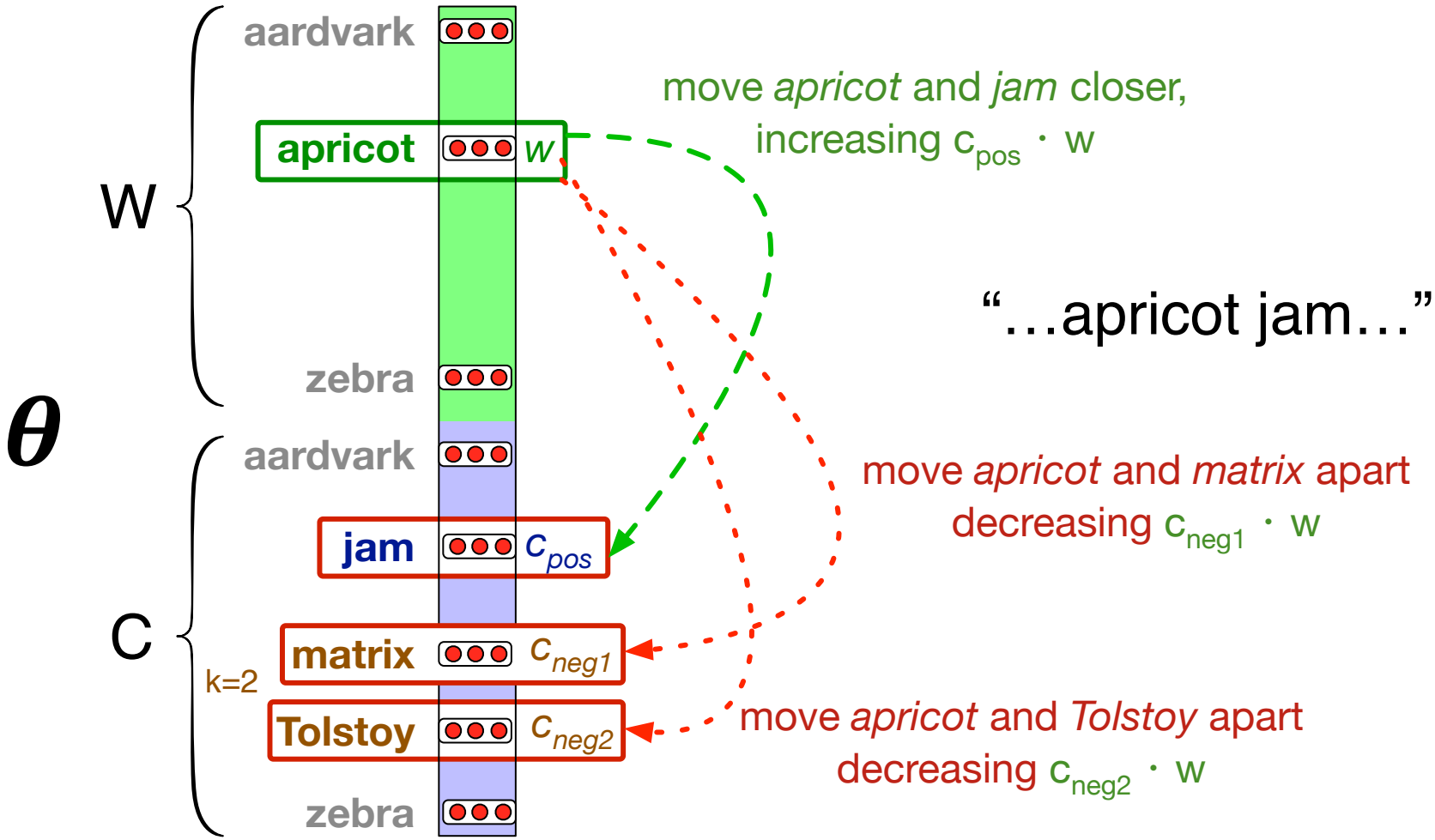
How to learn the classifier?

- Stochastic gradient descent!

We'll adjust the word weights to:

- make positive pairs $(w, c_{pos})$ more likely,
- make negative pairs $(w, c_{neg_i})$ less likely,
- across the entire dataset.

# Intuition for gradient descent



# Gradient descent reminder

At each step

- Move in the reverse direction to the gradient $\frac{\partial L}{\partial w}$ of the loss function $L$ wrt $w$
- Move an amount weighted by some small value $\eta$
- Higher learning rate means that we move faster

$$w^{t+1} = w^t - \eta \frac{\partial L}{\partial w}$$

# Derivatives of the skip-gram loss function

$$L = -\log \sigma(w \cdot c_{pos}) - \sum_{i=1}^{k} \log \sigma(w \cdot c_{neg_i})$$

It turns out that:

$$\frac{\partial L}{\partial c_{pos}} = (\sigma(w \cdot c_{pos}) - 1)w$$

$$\frac{\partial L}{\partial c_{neg}} = \sigma(w \cdot c_{neg})w$$

$$\frac{\partial L}{\partial w} = (\sigma(w \cdot c_{pos}) - 1)c_{pos} + \sum_{i=1}^{k} \sigma(w \cdot c_{neg_i})c_{neg_i}$$

# Updating the weights

We start with randomly initialised sets of $c$ and $w$ vectors, and update as follows:

$$c_{pos}^{t+1} = c_{pos}^{t} - \eta \left(\sigma \left(w^t \cdot c_{pos}^{t}\right) - 1\right) w^t$$

$$c_{neg}^{t+1} = c_{neg}^{t} - \eta \sigma \left(w^t \cdot c_{neg}^{t}\right) w^t$$

$$w^{t+1} = w^t - \eta \left( \left(\sigma \left(w^t \cdot c_{pos}\right) - 1\right) c_{pos} + \sum_{i=1}^{k} \left(\sigma \left(w^t \cdot c_{neg_i}\right)\right) c_{neg_i} \right)$$

# Two sets of embeddings!

Skipgram with negative sampling learns two sets of embeddings:

- Target embeddings $w$
- Context embeddings $c$

It is common to simply add them together, so that word $i$ is represented as $w_i + c_i$. Other options include concatenation.

# Summary: Skip-gram with negative sampling

- Start with random $d$-dimensional vectors as initial embeddings, collected in matrices $W$ and $C$
- Train a classifier based on embedding similarity:
  - Take a corpus and take pairs of words $(w, c_{pos})$ that co-occur as positive examples.
  - Take pairs of words $(w, c_{neg})$ that don't co-occur as negative examples.
  - Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance.
  - Finally, throw away the classifier code and keep the embeddings.

# Outline.

1 Introduction

2 Distributional semantics

3 Semantics with dense vectors

4 Properties of embeddings

# Properties of embeddings

## They capture similarity

| FRANCE | JESUS | XBOX | REDDISH | SCRATCHED | MEGABITS |
|---|---|---|---|---|---|
| 454 | 1973 | 6909 | 11724 | 29869 | 87025 |
| AUSTRIA | GOD | AMIGA | GREENISH | NAILED | OCTETS |
| BELGIUM | SATI | PLAYSTATION | BLUISH | SMASHED | MB/S |
| GERMANY | CHRIST | MSX | PINKISH | PUNCHED | BIT/S |
| ITALY | SATAN | IPOD | PURPLISH | POPPED | BAUD |
| GREECE | KALI | SEGA | BROWNISH | CRIMPED | CARATS |
| SWEDEN | INDRA | PSNUMBER | GREYISH | SCRAPED | KBIT/S |
| NORWAY | VISHNU | HD | GRAYISH | SCREWED | MEGAHERTZ |
| EUROPE | ANANDA | DREAMCAST | WHITISH | SECTIONED | MEGAPIXELS |
| HUNGARY | PARVATI | GEFORCE | SILVERY | SLASHED | GBIT/S |
| SWITZERLAND | GRACE | CAPCOM | YELLOWISH | RIPPED | AMPERES |

*Slide credit: Ronan Collobert*

# Properties of embeddings

They capture analogy

**Analogy task**: ***a*** *is to* ***b*** *as* ***c*** *is to* ***d***
The system is given words $a, b, c$, and it needs to find $d$.

*"apple" is to "apples" as "car"' is to ?*
*"man" is to "woman" as "king" is to ?*

**Solution**: capture analogy via vector offsets

$$a - b \approx c - d$$

$$man - woman \approx king - queen$$

$$d_w = \underset{d'_w \in V}{\text{argmax}}\, cos(a - b, c - d')$$

# Properties of embeddings

They capture analogy

**Analogy task**: ***a*** *is to* ***b*** *as* ***c*** *is to* ***d***
The system is given words $a, b, c$, and it needs to find $d$.

*"apple" is to "apples" as "car"' is to ?*
*"man" is to "woman" as "king" is to ?*

**Solution**: capture analogy via vector offsets

$$a - b \approx c - d$$

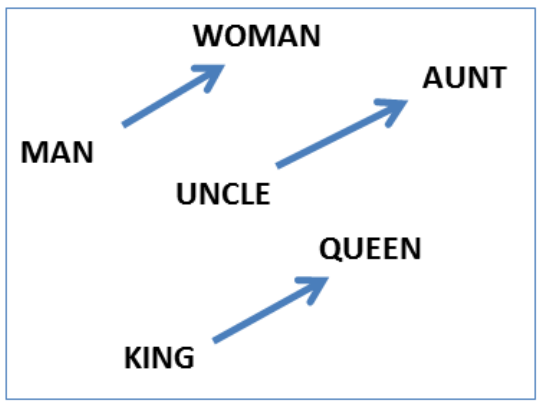
$$man - woman \approx king - queen$$

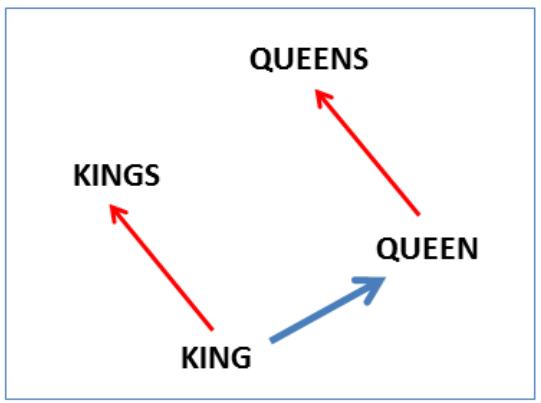
$$d_w = \underset{d'_w \in V}{\text{argmax}}\, cos(a - b, c - d')$$

# Properties of embeddings

Capture analogy via vector offsets

$$man - woman \approx king - queen$$




Mikolov et al. 2013. *Linguistic Regularities in Continuous Space Word Representations*

# Properties of embeddings

They capture a range of semantic relations

| Relationship | Example 1 | Example 2 | Example 3 |
|---|---|---|---|
| France - Paris | Italy: Rome | Japan: Tokyo | Florida: Tallahassee |
| big - bigger | small: larger | cold: colder | quick: quicker |
| Miami - Florida | Baltimore: Maryland | Dallas: Texas | Kona: Hawaii |
| Einstein - scientist | Messi: midfielder | Mozart: violinist | Picasso: painter |
| Sarkozy - France | Berlusconi: Italy | Merkel: Germany | Koizumi: Japan |
| copper - Cu | zinc: Zn | gold: Au | uranium: plutonium |
| Berlusconi - Silvio | Sarkozy: Nicolas | Putin: Medvedev | Obama: Barack |
| Microsoft - Windows | Google: Android | IBM: Linux | Apple: iPhone |
| Microsoft - Ballmer | Google: Yahoo | IBM: McNealy | Apple: Jobs |
| Japan - sushi | Germany: bratwurst | France: tapas | USA: pizza |

Mikolov et al. 2013. *Efficient Estimation of Word Representations in Vector Space*

# Word embeddings in practice

Word2vec is often used for pretraining in other tasks.

- It will help your models start from an **informed** position
- Requires only **plain text** - which we have a lot of
- Is very **fast** and easy to use
- Already **pretrained** vectors also available (trained on 100B words)

However, for best performance it is important to continue training, fine-tuning the embeddings for a specific task.

# Summary

- Dense, prediction-based embeddings have many useful properties.
- They are excellent at capturing similarity.
- They capture a suprising amount of analogical structure
- ...both semantic and morphological.
- In practice, you can use pretrained word embeddings to intialise otther models
- ... and there are useful libraries available for this, such as `gensim`

# Overall Summary

- Lexical semantics studies how the meanings of individual words can be represented and induced.
- We know certain things about how word meanings behave: relations and properties such as synonymy, hyponymy, gradedness.
- The distributional hypothesis tells us that meaning can be modelled as cooccurrence statistics
- Psychology tells us that meanings can be modelled as points in feature space
- Distributional semantics brings these ideas together.

# Overall Summary

- A word can be modelled as a vector embedding whose entries capture the cooccurrence of that word with others.
- The values of these entries can be learnt through counting cooccurrences.
- Or, better, word2vec gives a way to build embeddings via *predicting* cooccurrence.
- Learnt embeddings reflect key properties of word meanings such as similarity, analogy.
- Many resources for using these!

# Acknowledgement

*Some slides were adapted from Ann Copestake, Dan Jurafsky and Marek Rei*

# References I

[1] Charles J Fillmore and Collin F Baker.
Frame semantics for text understanding.
In *Proceedings of WordNet and other lexical resources workshop, NAACL*, volume 6, pages 59–64, 2001.

[2] Peter Gärdenfors.
*The Geometry of Meaning: Semantics Based on Conceptual Spaces*.
The MIT Press, January 2014.

[3] Felix Hill, Roi Reichart, and Anna Korhonen.
Simlex-999: Evaluating semantic models with (genuine) similarity estimation.
*Computational Linguistics*, 41(4):665–695, 2015.

# References II

[4] Theo M. V. Janssen and Thomas Ede Zimmermann.
Montague semantics, Nov 2011.

[5] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean.
Efficient Estimation of Word Representations in Vector Space.
*arXiv:1301.3781 [cs]*, September 2013.
arXiv: 1301.3781.

[6] Eleanor Rosch.
Cognitive representations of semantic categories.
*Journal of Experimental Psychology: General*, 104(3):192–233, 1975.
tex.ids: rosch19760101 place: US.

# References III

[7] Roger N. Shepard.
Toward a Universal Law of Generalization for Psychological Science.
*Science*, 237(4820):1317–1323, September 1987.

[8] Amos Tversky.
Features of similarity.
*Psychological Review*, 84(4):327–352, 1977.